

Departamento de Cs. e Ingeniería de la Computación

Universidad Nacional del Sur



Bases de Datos

Segundo Cuatrimestre de 2026

Proyecto Nº 3

Implementación de transacciones, stored procedures y triggers en un sistema de base de datos bancario

Ejercicios

1. El cajero automático (ATM) provee a los clientes del banco la posibilidad de realizar dos tipos de transacciones sobre sus cajas de ahorro: extracciones y transferencias. La operatoria de estas transacciones es la siguiente:

   - Una extracción corresponde al retiro de cierto monto de dinero por parte de uno de los titulares de la caja de ahorro. Una vez identificado el titular y la caja de ahorro mediante la tarjeta ingresada se solicita el monto a extraer y, si hay saldo disponible suficiente en la caja de ahorro, se decrementa el saldo de dicha caja de acuerdo al monto ingresado.
   - Una transferencia corresponde al movimiento de dinero hacia otra caja de ahorro del banco. Para efectuar dicho movimiento, es necesario identificar la caja de ahorro origen de los fondos y la caja de ahorro destino. La caja de ahorro origen se identifica mediante la tarjeta que es ingresada al cajero. La caja de ahorro destino se determinará luego de solicitar al cliente que ingrese un número de caja de ahorro destino. Observe que el destino de los fondos corresponde a una caja de ahorro, sin considerar al titular de dicha caja.

     Con la información de las cajas de ahorro origen y destino, se solicita el monto a transferir y, si hay saldo disponible suficiente y el número de caja de ahorro destino existe, se procede a decrementar el saldo de la caja de ahorro origen e incrementar el saldo de la caja de ahorro destino de acuerdo al monto ingresado. Por lo tanto, cuando se hace una transferencia en realidad se deben almacenar dos nuevas transacciones en la base de datos:
     - una transferencia asociada a la caja de ahorro origen y
     - un depósito (por el mismo monto) asociado a la caja de ahorro destino

   Como la base de datos puede ser accedida concurrentemente por varios ATM debe asegurarse que las transacciones se realicen de forma atómica y que los datos se bloqueen adecuadamente para garantizar la serializabilidad y evitar que la base de datos quede en un estado inconsistente.

   Mediante *stored procedures* implemente transacciones para:

   (a) **Realizar una transferencia entre dos cajas de ahorro** mediante un procedimiento con los siguientes parámetros:

   ```
   transferir(IN nro_atm INT, IN tarjeta_origen BIGINT, IN destino INT, IN
       monto DECIMAL(16,2), OUT resultado VARCHAR(100))
   ```

   donde `nro_atm` es el identificador del cajero automático, `tarjeta_origen` es el número de tarjeta asociado a la caja de ahorro origen de los fondos, `destino` es el número de la caja de ahorro destino y `monto` es la cantidad de dinero a transferir.

   (b) **Realizar una extracción de una caja de ahorro** mediante un procedimiento con los siguientes parámetros:

   ```
   extraer(IN nro_atm INT, IN tarjeta BIGINT, IN monto DECIMAL(16,2), OUT
       resultado VARCHAR(100))
   ```

donde `nro_atm` es el código del cajero automático, `tarjeta` es el número de tarjeta del titular que retira dinero y `monto` es el importe a extraer.

Ambos *stored procedures* deberán devolver en el parámetro de salida `resultado` el estado final de la transacción, indicando si la operación se realizó con éxito o el motivo de su rechazo (por ejemplo: saldo insuficiente, tarjeta no existente, caja de ahorro destino inexistente, código de ATM inválido, etc.).

Concurrencia y Atomicidad: Debe asegurarse de que las operaciones se ejecuten dentro de transacciones que garanticen atomicidad y aislamiento. Para evitar anomalías de concurrencia al leer y actualizar el saldo, deberá utilizar bloqueos exclusivos en las lecturas correspondientes (utilizando `SELECT ... FOR UPDATE`). En caso de cualquier error imprevisto (manejado mediante un `EXIT HANDLER FOR SQLEXCEPTION`), se deberá deshacer la transacción (`ROLLBACK`) y devolver un mensaje de error.

Importante: Dado que el campo *nro_trans* de la tabla *Transaccion* es de tipo `AUTO_INCREMENT`, deberá utilizar la función `LAST_INSERT_ID()` para recuperar el número generado automáticamente al insertar una nueva transacción. Recuerde que para registrar la fecha y hora de la operación debe utilizar las funciones de MySQL `CURDATE()` y `CURTIME()` para recuperar la fecha y hora actual.

2. **Implemente un trigger** asociado a la tabla *prestamo* que se active automáticamente cuando se inserta un nuevo préstamo y cargue todos los pagos (cuotas) asociados al mismo en la tabla *pago*. La cantidad de pagos a generar estará determinada por el atributo *cant_meses* del préstamo insertado. La fecha de vencimiento (*fecha_venc*) de cada pago deberá calcularse a partir de la fecha de creación del préstamo incrementando un mes por cada cuota sucesiva: la cuota 1 vencerá un mes después de la fecha del préstamo, la cuota 2 dos meses después, y así sucesivamente hasta la última cuota. Para calcular las fechas de vencimiento puede utilizar la función de MySQL `DATE_ADD(NEW.fecha, INTERVAL nro_pago MONTH)` (ver más funciones para manejo de fechas y tiempo en el apéndice A). Las fechas de pago (*fecha_pago*) de todas las cuotas generadas deberán dejarse en `NULL`, ya que serán actualizadas cuando el cliente efectúe el pago correspondiente.

3. Implemente los siguientes *stored procedures* para llevar a cabo consultas y operaciones comunes del sistema bancario:

   (a) **Últimos movimientos:**

   `ultimos_movimientos(IN tarjeta BIGINT)`

   Devuelve una tabla con las últimas 15 transacciones realizadas sobre la caja de ahorro asociada a la tarjeta ingresada. El listado deberá ordenarse por fecha descendente y hora descendente, e incluir las siguientes columnas:

   - `fecha`: fecha de la transacción.
   - `hora`: hora de la transacción.
   - `tipo`: tipo de transacción (*'deposito'*, *'extraccion'*, *'transferencia'* o *'debito'*).
   - `monto`: importe de la operación. En caso de transacciones que decrementen el saldo de la caja de ahorro (extracciones, transferencias y débitos), el monto deberá mostrarse con signo negativo (por ejemplo: `IF(UPPER(tipo) = 'DEPOSITO', monto, (-1) * monto)`).
   - `cod_caja`: código del cajero automático o ventanilla donde se efectuó (o `NULL` si no aplica).
   - `destino`: número de caja de ahorro destino en caso de ser una transferencia (o `NULL` en otro caso).

(b) **Movimientos por período:**

`movimientos_por_periodo(IN tarjeta BIGINT, IN desde DATE, IN hasta DATE)`

Devuelve una tabla con la misma estructura de columnas que `ultimos_movimientos`, conteniendo todas las transacciones realizadas sobre la caja de ahorro de la tarjeta especificada cuyas fechas estén comprendidas entre `desde` y `hasta` (inclusive), ordenadas por fecha descendente y hora descendente.

(c) **Creación de préstamos:**

```
crear_prestamo(IN nro_cliente INT, IN legajo INT, IN monto DECIMAL(10,2),
               IN cant_meses INT, OUT resultado VARCHAR(100))
```

Permite a un empleado registrar un nuevo préstamo para un cliente. Deberá implementar las siguientes reglas y validaciones:

- Controlar que el cliente exista.
- Controlar que el cliente **no tenga préstamos vigentes**. Se considera que un cliente posee un préstamo vigente si tiene algún préstamo previo con cuotas pendientes de pago (es decir, cuotas con `fecha_pago IS NULL`). Si posee un préstamo vigente, la operación se rechazará devolviendo un mensaje explicativo en `resultado`.
- Controlar que el monto solicitado no supere el monto máximo (`monto_sup`) ni sea menor que el monto mínimo (`monto_inf`), y que la cantidad de meses (`cant_meses`) coincida con un período válido en la tabla **Tasa_prestamo**.
- En caso afirmativo, obtener la tasa de interés correspondiente de **Tasa_prestamo** y calcular:
  - El interés total del préstamo: $\texttt{interes} = \dfrac{\texttt{monto} \times \texttt{tasa} \times \texttt{cant_meses}}{1200}$
  - El valor de cada cuota: $\texttt{valor_cuota} = \dfrac{\texttt{monto} + \texttt{interes}}{\texttt{cant_meses}}$
- Insertar el nuevo préstamo en la tabla **Prestamo** con fecha actual (`CURDATE()`), asignando el cliente y el legajo del empleado ingresados.
  Observación: Al insertar el préstamo, el trigger implementado en el ejercicio 2 se disparará de forma automática, cargando las cuotas en la tabla **Pago**.
- Informar en el parámetro de salida `resultado` si el préstamo fue creado exitosamente o el motivo por el cual no pudo otorgarse.

(d) **Pago de cuotas:**

```
pagar_cuota(IN nro_prestamo INT, IN nro_pago INT, OUT resultado
            VARCHAR(100))
```

Permite registrar el pago de una cuota de un préstamo:

- Verificar que el préstamo exista.
- Verificar que la cuota (`nro_pago`) pertenezca a dicho préstamo.
- Verificar que la cuota no esté ya pagada (su `fecha_pago` debe ser `NULL`).
- Si las validaciones son exitosas, actualizar `fecha_pago = CURDATE()` para la cuota correspondiente y devolver un mensaje de confirmación en `resultado`. De lo contrario, reportar el error en dicho parámetro.

(e) **Listado de clientes morosos:**

`listado_morosos(INT cant_cuotas)`

Devuelve una tabla con la información de los clientes que adeudan **al menos** `cant_cuotas` (parámetro) de su préstamo vigente. Es decir, poseen una cantidad de cuotas impagas (fecha de vencimiento es menor a la fecha actual y fecha de pago es NULL) que es mayor o igual a `cant_cuotas`. La tabla resultante contendrá las siguientes columnas:

- `nro_cliente`: número identificador del cliente.
- `tipo_doc`: tipo de documento del cliente.
- `nro_doc`: número de documento del cliente.
- `nombre`: nombre del cliente.
- `apellido`: apellido del cliente.
- `nro_prestamo`: número del préstamo en mora.
- `monto`: monto original del préstamo.
- `cant_meses`: plazo acordado en meses.
- `valor_cuota`: importe mensual de la cuota.
- `cant_cuotas_atrasadas`: cantidad de cuotas vencidas e impagas.

Consideraciones sobre los privilegios

Las sentencias SQL para la creación de todos los *stored procedures* y del trigger deberán definirse dentro de la base de datos *banco* e incluirse en el archivo ***"banco.sql"***.

Además, deberá extender los privilegios de los usuarios correspondientes:

- El usuario ***atm*** deberá tener privilegios de ejecución (`EXECUTE`) sobre los procedimientos vinculados a la operatoria de cajero:
  - `transferir`
  - `extraer`
  - `ultimos_movimientos`
  - `movimientos_por_periodo`
- El usuario ***empleado*** deberá tener privilegios de ejecución (`EXECUTE`) sobre los procedimientos utilizados en la administración bancaria:
  - `crear_prestamo`
  - `pagar_cuota`
  - `listado_morosos`

A Funciones de MySQL para el manejo de fechas y tiempo

MySQL provee funciones para manipular fechas y tiempo que resultan muy útiles para el desarrollo de este proyecto. En general estas funciones se pueden consultar al servidor mediante la sentencia SQL select y devuelven una tabla con una única celda que contiene el resultado (más información ver sección 12.7 del manual de MySQL). A continuación se muestran ejemplos de algunas de las funciones disponibles al invocarlas desde el cliente *mysql.exe*.

- ```
  mysql> SELECT CURDATE();
  +------------+
  | CURDATE()  |
  +------------+
  | 2021-09-27 |
  +------------+
  1 row in set (0.03 sec)
  ```

  Devuelve la fecha actual en la forma: año-mes-día

- ```
  mysql> SELECT CURTIME();
  +-----------+
  | CURTIME() |
  +-----------+
  | 13:09:37  |
  +-----------+
  1 row in set (0.00 sec)
  ```

  Devuelve la hora actual .

- ```
  mysql> SELECT NOW();
  +---------------------+
  | NOW()               |
  +---------------------+
  | 2021-09-27 13:09:45 |
  +---------------------+
  1 row in set (0.00 sec)
  ```

  Devuelve la fecha y hora actual .

- ```
  mysql> SELECT DAYOFWEEK('2021-09-26');
  +-------------------------+
  | DAYOFWEEK('2021-09-26') |
  +-------------------------+
  |                       1 |
  +-------------------------+
  1 row in set (0.02 sec)
  ```

  El número devuelto representa el día de la semana correspondiente a la fecha dada. 1 = domingo, 2 = lunes, 3 = martes, ... , 7 = sábado.

- ```
  mysql> SELECT TIMEDIFF('13:40:00', '08:05:00');
  +----------------------------------+
  | TIMEDIFF('13:40:00', '08:05:00') |
  +----------------------------------+
  | 05:35:00                         |
  +----------------------------------+
  1 row in set (0.00 sec)
  ```

  `TIMEDIFF(h1, h2)` devuelve la diferencia de tiempo entre dos horas `h1` y `h2`. `h1` debe ser mayor que `h2`, sino devolverá un número negativo.

  `TIMEDIFF` también se puede usar para calcular la diferencia de tiempo entre dos fechas:

  ```
  mysql> SELECT TIMEDIFF('2021-09-27 01:01:01','2021-09-26 23:59:59');
  +-----------------------------------------------------------+
  | TIMEDIFF('2021-09-27 01:01:01','2021-09-26 23:59:59') |
  +-----------------------------------------------------------+
  | 01:01:02                                                  |
  +-----------------------------------------------------------+
  1 row in set (0.00 sec)
  ```

- ```
  mysql> SELECT TIME_TO_SEC('1:00:00');
  +------------------------+
  | TIME_TO_SEC('1:00:00') |
  +------------------------+
  |                   3600 |
  +------------------------+
  1 row in set (0.00 sec)
  ```

  Devuelve la cantidad de segundos que representa el tiempo dado.